

EC8207: Applied Big Data Engineering

Mini Project Assessment

The "Smart City" Traffic & Congestion System

Group Members:

Prabuddhika R.P.H	EG/2020/4116
Virajani M.Y.	EG/2020/4254
Wahalathanthri W.A.S.	EG/2020/4257

1 Introduction

Urban Traffic Congestion is among the most pressing issues facing modern urban centers. The Smart City Traffic Congestion System is an application that analyses traffic flow on four key junctions Junction A, Junction B, Junction C, and Junction D in Colombo, Sri Lanka. The application generates simulated IoT sensor data, analyses it in real time using state of the art stream processing technologies and provides actionable information for deployment by traffic law enforcement. This data processing architecture caters for three key characteristics of big data, velocity, volume and variety.

2 Selected Tools

Each tool in the pipeline has been picked on the basis of its technical compatibility with the system's requirements.

2.1 Message Broker - Apache Kafka 7.5.0

Kafka is used as the ingestion and buffering layer. Messages are produced at a rate of 1 JSON message, second, junction in the topic traffic data on kafka9092. Kafka's log-based storage ensures durability and replay of messages, enabling Spark to recover from failures without losing any data. The setting `KAFKA_AUTO_CREATE_TOPICS_ENABLE`- 'true' in `docker-compose.yml` saves us from having to create topics manually. Importantly, Kafka adds a broker generated timestamp to each message that Spark relies upon as the official event time. A more straightforward queue like RabbitMQ would have been inadequate since it cannot replay messages once consumed. For traffic data, there is a single partition because it is enough for four sensors sending four messages every second. The keys used in messages are `sensor_id`, ensuring that data related to the same junctions appear together. Having a partition per junction in a live environment allows us to process the messages in parallel using Spark.

2.2 Processing Engine - Apache Spark 3.4.1 Structured Streaming

Spark Structured Streaming forms the heart of `traffic_streaming.py`. It ingests data from Kafka using spark SQL Kafka connector, performs two-minute sliding windows with 30 second watermark, and calculates `avg_speed`, `avg_vehicle_count`, `reading_count`, and `congestion_index` on each window for each sensor. The data is exported to PostgreSQL once every 15 seconds using

foreachBatch operation through JDBC write. At the same time, Parquet data is stored on disk for future analysis. Spark runs locally with configuration of local and number of shuffle partitions as 4. Spark's native support for event-time windows and DataFrames made it an ideal engine for this purpose.

2.3 Workflow Orchestration - Apache Airflow 2.7.0

Airflow handles the execution of the batch pipeline as coded in `traffic_batch_dag.py`. The smart city traffic report DAG runs on a daily schedule of 01:00 hours and `catchup=False` to prevent backfilling of missed batches. There are three sequential steps defined as `PythonOperators`, `extract_daily_traffic` to export data from PostgreSQL into CSV format, `analyze_peak_hours` to find the peak hours by group by and `idxmax` method of `pandas`, and `generate_report` to create a report with plots and text via `matplotlib/seaborn` libraries. No intermediate database is used for exchanging data between operators; it all happens via XCom. Default args, `retries=2`, `retry delay=5` minutes.

2.4 Relational Storage - PostgreSQL 14

PostgreSQL database contains three tables, those, traffic history all window aggregations, critical traffic alerts all windows with `avg_speed < 10` km/h, and daily reports metadata about nightly batch job. Indexes `idx_history_sensor_time`, `idx_sensor_time`, and `idx_window_start` improve performance for hour-based aggregation queries and date range queries. The daily reports table contains an `ON CONFLICT DO UPDATE` statement to make idempotent updates possible when DAG runs are repeated. Two PostgreSQL databases are run inside Docker, each on a different port traffic data on 5432 and Airflow metadata on 5433, to avoid a schema collision.

2.5 Live Dashboard - Streamlit and Plotly

This Streamlit dashboard offers a pure-Python way to build a web application, without needing to write any JavaScript code. All fetch functions have the `@st.cache_data-ttl=10` decorator that makes sure that the database is only queried at least once in 10 seconds, avoiding the connection exhaustion issue. Interactive charts are plotted using Plotly Express and Plotly Graph Objects as interactive area plots, line plots, and stacked bar plots. Junction selector in the sidebar allows filtering all queries by a junction.

Table 1: summarizes the justification for each tool

Tool	Version	Role	Key Justification
Apache Kafka	7.5.0	Message broker	Message replay, decoupling, broker timestamps
Apache Spark	3.4.1	Stream processing	Event time windowing, Kafka integration, JDBC output
Apache Airflow	2.7.0	Orchestration	DAG scheduling, XCom, retry logic
PostgreSQL	14	Storage	Structured schema, complex aggregation queries
Streamlit	Latest	Dashboard	Pure Python UI, cache TTL, Plotly integration
Docker Compose	Latest	Infrastructure	Reproducibility, health checks, bridge networking

3 Event Time vs. Processing Time Handling

The event time refers to the timestamp of the data point as created at the sensor. Processing time is the wall clock time during which Spark processes the message. Ideally, both will be the same, but any delay due to networking issues can cause huge variance.

The Kafka DataFrame already has its own timestamp column which in `traffic_streaming.py` is mapped to the event time column. The event time is used as the watermark and partitioning criteria in `withWatermark` and window functions. However, there is an important difference between Kafka's timestamp and the JSON message's ISO timestamp. The former is used by the system because it prevents possible skew from producers' clocks to mess up the window boundaries. A message generated at 08:59:55 needs to go into 08:58-09:00 window rather than 09:00-09:02 if processed at 09:00:05. Otherwise, the Airflow would use the wrong statistics to allocate police force.

Table 2: Event Time vs Processing Time - Design Decisions

Aspect	This System's Approach	Rationale
Time source	Kafka broker timestamp - event time	Immune to producer clock skew, broker assigned is authoritative
Window size	2-minute tumbling window	Balances granularity with processing overhead
Watermark	30 seconds	Tolerates network delays, cleanly closes windows
Trigger interval	15 seconds - processing time	Controls micro batch frequency only, does not affect windowing
Late data policy	Dropped after watermark expires	Acceptable given one second sensor publishing frequency

4 Privacy Ethics and Data Governance

4.1 Nature of Data Collected

As of now, the technology uses only two pieces of information on a per junction basis, vehicle count and avg speed. These are aggregate levels of information and involve neither personal ID numbers nor any other form of identification such as a license plate number or GPS coordinates. This can be seen in the data model described in `init.sql` file, and further demonstrated by the payload of the producer in `traffic_producer.py`.

As such, the technology is privacy preserving on the collection side, but certain things need to be considered when discussing its ethical implications.

4.2 Surveillance Risks in Smart City Deployment

- **Spatiotemporal correlation:** Tracking more junctions over an extended period allows for the inference of mobility patterns at a societal level. With enhanced resolution of sensors that can detect vehicles individually, mobility patterns of individuals can be inferred even without explicitly identifying individuals.

- **Function creep:** The PostgreSQL database keeps records of events indefinitely. hence, a longitudinal record is created that can be exploited. Traffic control systems have been misappropriated in the past for surveillance purposes without explicit public consent.
- **Deployment of police as profiling mechanism:** Deploying police to specific locations based on the algorithm's recommendation may create disparities among communities depending on their location in terms of traffic volume. If the congestion index has a spatiotemporal correlation with socio economic factors, such communities would be adversely affected.
- **Algorithmic accountability:** The congestion index calculation and criteria for allocating police units are value judgments encoded in the program, which are nontransparent and nonaccountable.

4.3 Data Governance Framework

In order to deploy this solution in Colombo responsibly, the following governance considerations need to be addressed,

- **Data minimization-** Keep only windowed aggregations but no loggings at an individual message level, implement automatic data cleanup of the traffic history records older than 90 days via a specific Airflow maintenance DAG.
- **Legal compliance-** Define a clear legal basis for the solution implementation using the Sri Lanka Personal Data Protection Act-2022, develop a public data processing notice.
- **Least privilege model-** Implement least privilege-based user roles in the database where Spark is an INSERTer while Streamlit is only SELECT, Airflow is an INSERTer for daily_reports, no hardcode of credentials in source files.
- **Security measures-** Log all database accesses with timestamps and identities. Develop an incident response procedure with notifications to the corresponding authorities in case of a breach.
- **Ethical review-** Obtain a prior ethical review of the system prior to its implementation including input from civil society, legal experts, and community members from the monitored areas, continue stakeholder involvement to address potential inequity concerns.

- Public scrutiny- Publicize the algorithm used to calculate the congestion index and deployment threshold. Also, publish the generated Airflow reports for transparency purposes.

5 Conclusion

The Smart City Traffic Congestion System represents a full scale, business oriented big data pipeline solution that tackles issues of velocity, volume, and variety. Kafka offers resilient data ingestion, with built in capabilities for data message replay. Spark Structured Streaming offers correct event time windowing to guarantee accurate peak hour analysis free from processing errors associated with late arrival of messages. Airflow is used to automatically create daily batch reports, while PostgreSQL combined with Streamlit connects raw sensor data to police operational decision making in a containerized environment provided by Docker Compose. Ethical use of this system requires access control based on user roles, data deletion after an expiration date, clear documentation of algorithms, and constant community involvement.